

การออกแบบบริการ (Service Design)

Challenges & Solutions: ฟีเจอร์ Team Chat สำหรับระบบ Task Board

1. บทนำ (Introduction)

ฟีเจอร์ Team Chat ถูกออกแบบในรูปแบบ Microservices แยกจาก Task Service เพื่อรองรับระบบแชทแบบ Real-time พร้อมฟีเจอร์แจ้งเตือน, ค้นหาประวัติการสนทนา และรองรับการส่งไฟล์ เป้าหมายหลักคือการสร้างระบบที่มีความสามารถในการขยายตัว (Scalable), ดูแลรักษาง่าย (Maintainable) และมีประสิทธิภาพสูง (High Performance)

2. Challenges และวิธีแก้ไข (Solutions)

2.1 การแยก Chat Service ออกจาก Task Service

Challenge:

- ระบบแชทและระบบจัดการงานมีพฤติกรรมการใช้งานและความต้องการในการขยายตัวแตกต่างกันมาก
- การรวมบริการทั้งสองไว้ในตัวเดียวทำให้ระบบซับซ้อน และยากต่อการพัฒนาและดูแลรักษา

Solution:

- ออกแบบ Chat Service เป็น Microservice แยกต่างหากจาก Task Service
- ช่วยให้สามารถพัฒนาและ Deploy แยกกันได้ ไม่ต้องพึ่งพากันมาก ลดความซับซ้อน
- เพิ่มความยืดหยุ่นและสามารถปรับแต่ง Chat Service ให้เหมาะกับงาน Real-time ได้โดยเฉพาะ

2.2 การรองรับระบบแชทแบบ Real-time

Challenge:

- ต้องรองรับการส่งข้อความจำนวนมากพร้อมกันแบบ Real-time โดยมีความหน่วงต่ำ
- ต้องการการสื่อสารแบบสองทาง (Full-duplex) ระหว่าง Client กับ Server

Solution:

- เลือกใช้ WebSocket (เช่น Socket.IO) แทนการ Polling เนื่องจากมี Latency ต่ำและประสิทธิภาพสูง
- ใช้ Node.js ซึ่งรองรับ Non-blocking Event-driven Architecture เหมาะสำหรับ Real-time Application
- ใช้ Redis Pub/Sub เพื่อซิงค์สถานะและข้อความระหว่างหลาย instance ของ Chat Service

1. Client เชื่อมต่อ WebSocket กับ Chat Service
2. Client ส่งคำขอ Join ห้อง (boardId)
3. Client ส่งข้อความ (Message)
4. Chat Service ประมวลผลและบันทึกข้อมูลใน MongoDB
5. Chat Service ส่ง Event ไปยัง Kafka
6. Notification Service ดึง Event จาก Kafka ส่ง Notification
7. Chat Service กระจายข้อความ Real-time ไปยัง Client ทุกคนในห้องเดียวกัน

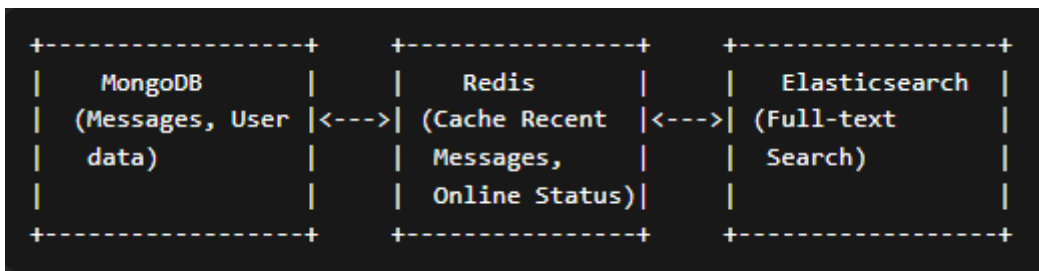
2.3 การจัดการข้อความและประวัติการสนทนา

Challenge:

- ต้องจัดเก็บข้อความจำนวนมากอย่างมีประสิทธิภาพ และรองรับการค้นหาข้อความย้อนหลัง
- ระบบต้องสามารถรองรับการเขียนข้อมูลที่มี Throughput สูง และโครงสร้างข้อมูลต้องยืดหยุ่น

Solution:

- ใช้ MongoDB เป็นฐานข้อมูลหลัก เนื่องจากรองรับ schema ยืดหยุ่นและมีประสิทธิภาพในการเขียนข้อมูลจำนวนมาก
- ใช้ Redis Cache เพื่อเก็บข้อความล่าสุดและสถานะผู้ใช้งาน ช่วยเพิ่มความเร็วในการเข้าถึงข้อมูลแบบ Real-time
- ใช้ Elasticsearch (optional) สำหรับฟังก์ชันค้นหาข้อความแบบ Full-text ที่รวดเร็ว



2.4 การแจ้งเตือนและการส่งข้อความหลายผู้ใช้ในเวลาเดียวกัน

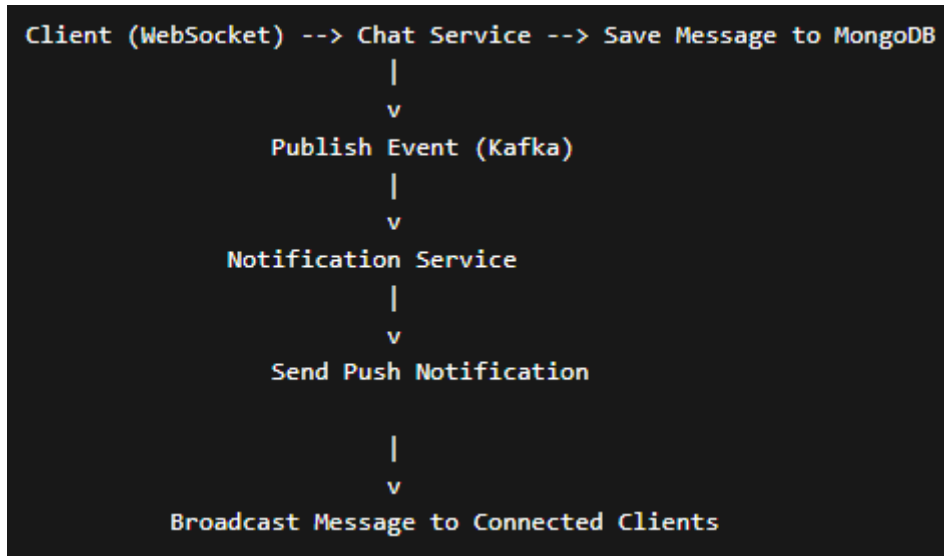
Challenge:

- ต้องกระจายข้อความและแจ้งเตือนไปยังผู้ใช้ในห้องเดียวกันแบบพร้อมกัน
- ต้องเพิ่มความน่าเชื่อถือและลดความล่าช้าในการส่งข้อมูล

Solution:

- ใช้ Message Broker เช่น Apache Kafka เพื่อสนับสนุน Event-driven Architecture

- ให้ Notification Service และ Chat Service สื่อสารกันผ่าน Event Broker ช่วยลดการพึ่งพาตรงกันระหว่างบริการ
- เพิ่มความทนทานและความน่าเชื่อถือในการส่งข้อมูลด้วยระบบ Messaging Queue



2.5 การจัดเก็บและส่งไฟล์ขนาดใหญ่

Challenge:

- ระบบต้องรองรับการส่งและจัดเก็บไฟล์ขนาดใหญ่จำนวนมากอย่างมีประสิทธิภาพ
- ต้องสามารถขยายระบบเก็บไฟล์ได้ง่ายในอนาคต

Solution:

- ใช้ Object Storage เช่น Amazon S3 หรือ MinIO ซึ่งรองรับการเก็บไฟล์ขนาดใหญ่และขยายระบบได้ง่าย
- แยก File Service เพื่อจัดการการอัปโหลดและดาวน์โหลดไฟล์โดยเฉพาะ

2.6 การดูแลรักษาและการขยายตัวของระบบ

Challenge:

- ระบบที่รวมทุกฟีเจอร์ในบริการเดียวจะยากต่อการแก้ไขและขยายขนาด
- ต้องการให้สามารถอัปเดตบริการบางส่วนโดยไม่กระทบทั้งระบบ

Solution:

- ใช้สถาปัตยกรรมแบบ Microservices ที่แยกแต่ละบริการตามหน้าที่เฉพาะ

- เลือกใช้เทคโนโลยีที่เหมาะสมสำหรับแต่ละบริการ เช่น Redis สำหรับ Cache, Kafka สำหรับ Event Messaging
- สามารถปรับขนาด (Scale) แต่ละบริการได้แยกตามปริมาณงาน



4. สรุป

การออกแบบฟีดแบ็ก Team Chat ในรูปแบบ Microservices พร้อมใช้ WebSocket, Kafka, MongoDB, Redis, และ Object Storage สามารถแก้ไขปัญหาเรื่อง Latency, Scalability และ Maintainability ได้อย่างมีประสิทธิภาพระบบนี้เหมาะสำหรับรองรับการใช้งานของผู้ใช้จำนวนมาก พร้อมฟีดแบ็ก Real-time Chat ที่เสถียรและรวดเร็ว